数据结构

查找

考试大纲

六、查找

（一）查找的基本概念

（二）顺序查找法

（三）分块查找法

（四）折半查找法

（五）树型查找

1.二叉树搜索树

2.平衡二叉树

3.红黑树

（六）B 树及其基本操作、B+ 树的基本概念

（七）散列（Hash）表

（八）字符串模式匹配

（九）查找算法的分析及应用

查找的基本概念与衡量标准

查找 (Searching)： 在数据集合中寻找满足某种条件的记录。

平均查找长度 (ASL, Average Search Length)： 查找算法效率最核心指标。

$$ASL = \sum_{i=1}^{n} p_i c_i$$

其中 p_i 是查找第 i 个记录的概率，c_i 是查找到该记录所需的比较次数。

通常考虑查找成功、查找失败两种情况下的ASL

顺序查找 (Sequential Search)

适用场景： 线性表，无论是顺序存储还是链式存储。

> **优化技巧：哨兵 (Sentinel)**
>
> 将 ST.elem[0] 设置为查找关键字，免去每步判断数组越界，效率提高约 1/4。

场景	成功 ASL
等概率	(n+1)/2
不等概率	按概率降序排列

失败 ASL：n+1 (哨兵模式下)。

分块查找 (Block Search)

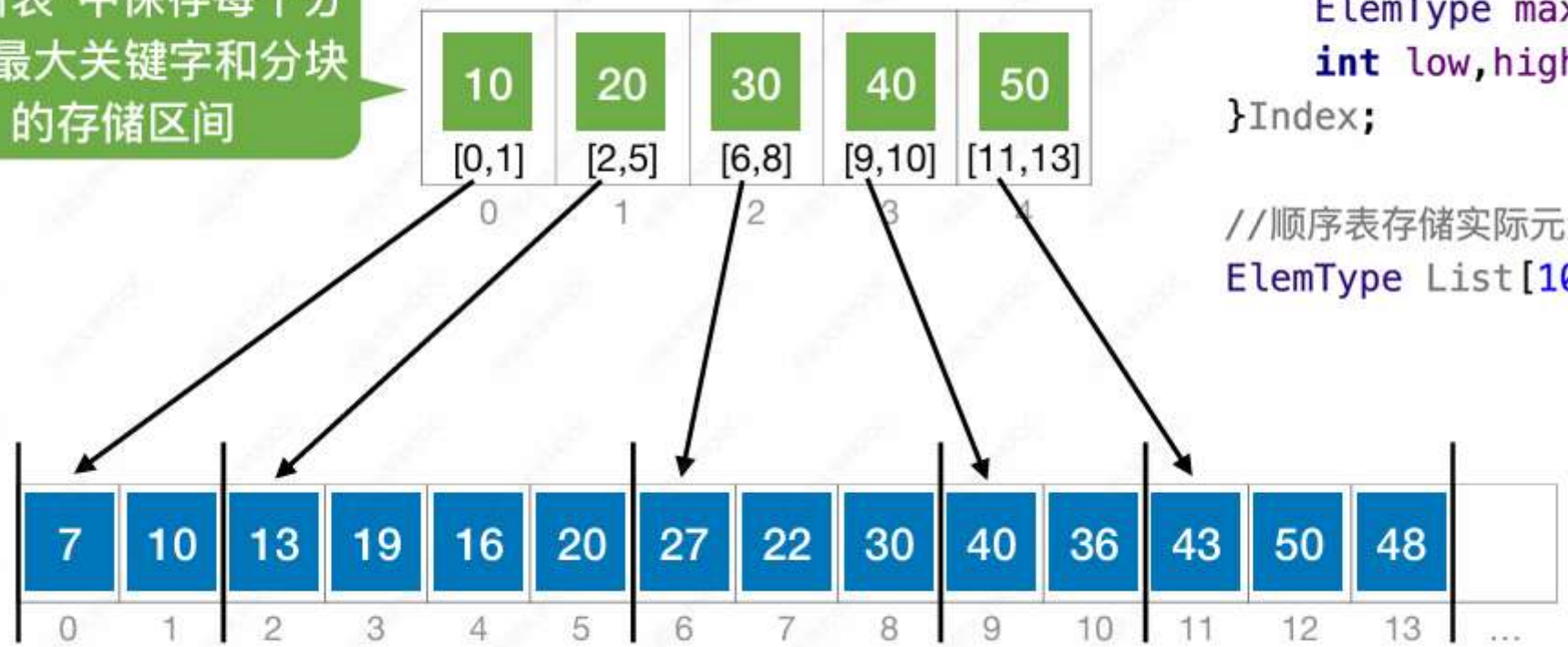
又称：索引顺序查找。 结合了顺序查找和折半查找的优点。

- **结构：** 索引表 (有序) + 主表 (块内无序，块间有序)。
- **ASL 公式：** $ASL = L_I + L_S$(索引表查找长度 + 块内查找长度)。
- 若分 n 个元素为 s 块，每块 b 个，均采用顺序查找，则 ASL = (s+1)/2 + (b+1)/2。

当 $s = \sqrt{n}$ 时，ASL 取得最小值 $\sqrt{n} + 1$ 。

分块查找的算法思想

“索引表”中保存每个分块的最大关键字和分块的存储区间

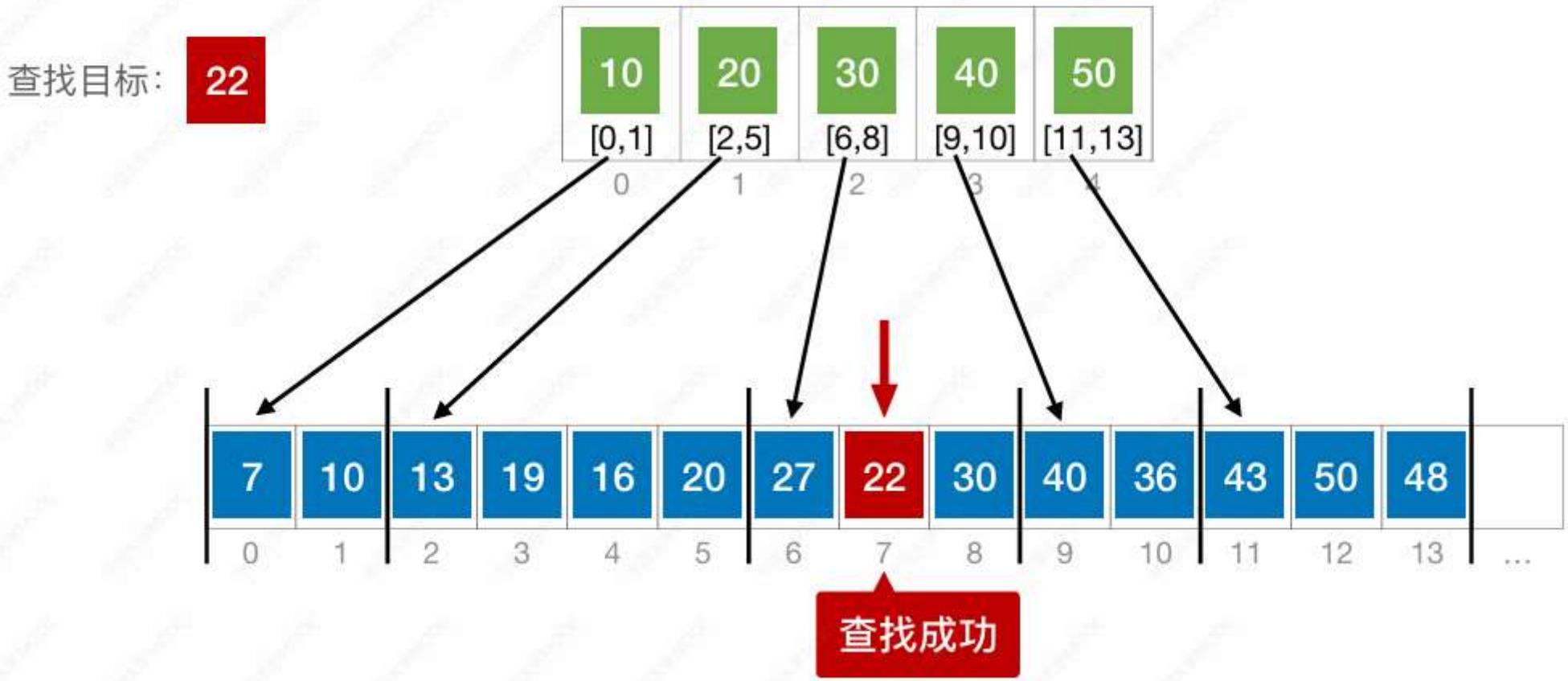



```c
//索引表
typedef struct {
    ElemType maxValue;
    int low,high;
}Index;

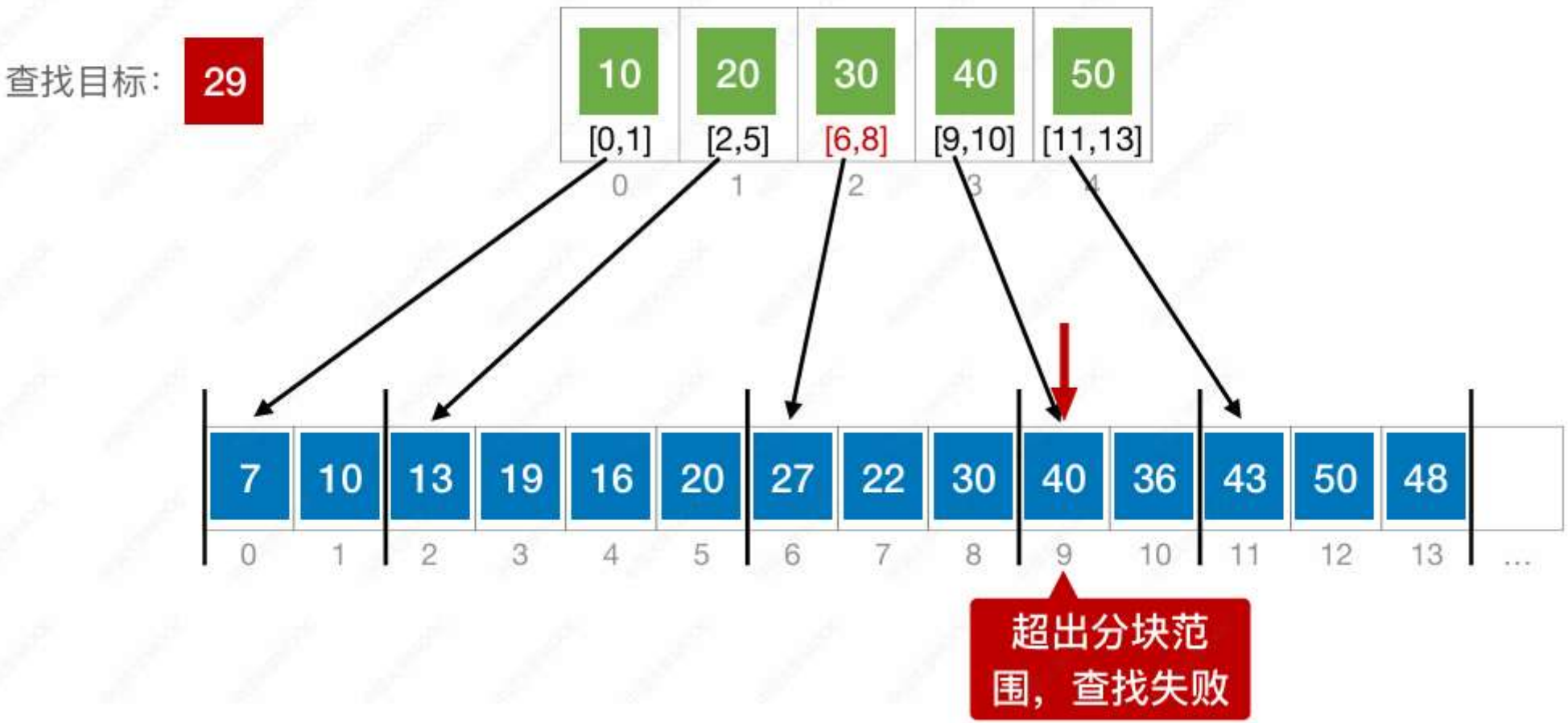
//顺序表存储实际元素
ElemType List[100];
```

特点：块内无序、块间有序

分块查找的算法思想


查找目标：
22
10
20
30
40
50
[0,1]
[2,5]
[6,8]
[9,10]
[11,13]
0
1
2
3
4
7
10
13
19
16
20
27
22
30
40
36
43
50
48
0
1
2
3
4
5
6
7
8
9
10
11
12
13
...
查找成功

分块查找的算法思想

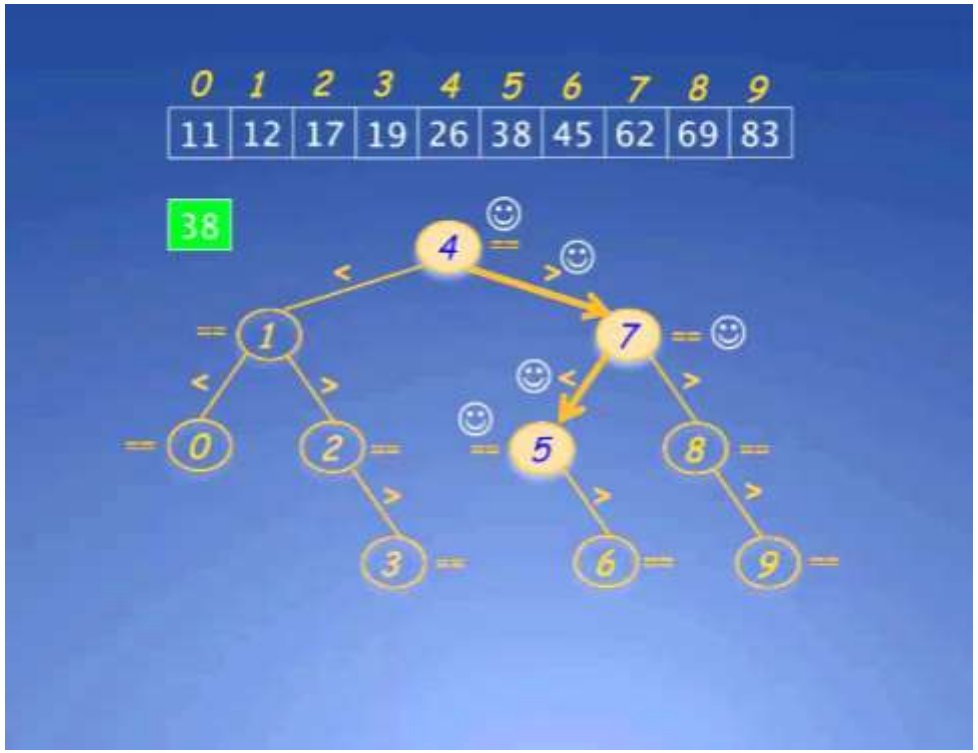

查找目标：
29
10
20
30
40
50
[0,1]
[2,5]
[6,8]
[9,10]
[11,13]
0
1
2
3
4
7
10
13
19
16
20
27
22
30
40
36
43
50
48
0
1
2
3
4
5
6
7
8
9
10
11
12
13
...
超出分块范围，查找失败

折半查找 (Binary Search)

核心前提：

1. 必须采用**顺序存储**结构。
2. 关键字必须**按序排列**。

> 时间复杂度：$O(\log_2 n)$



查找判定树 (Decision Tree)

折半查找的过程可以用一棵平衡二叉树表示。

特征	描述
圆节点	内部节点，代表查找成功的路径
矩形节点	外部节点/失败节点，代表查找失败的区间
高度	$\lfloor \log_2 n \rfloor + 1$

🚨 **要点：** 成功 ASL 计算需统计各层圆节点；失败 ASL 计算需统计矩形节点所在层数并计算平均值。

二叉搜索树 (BST)

定义： 左子树 < 根 < 右子树。

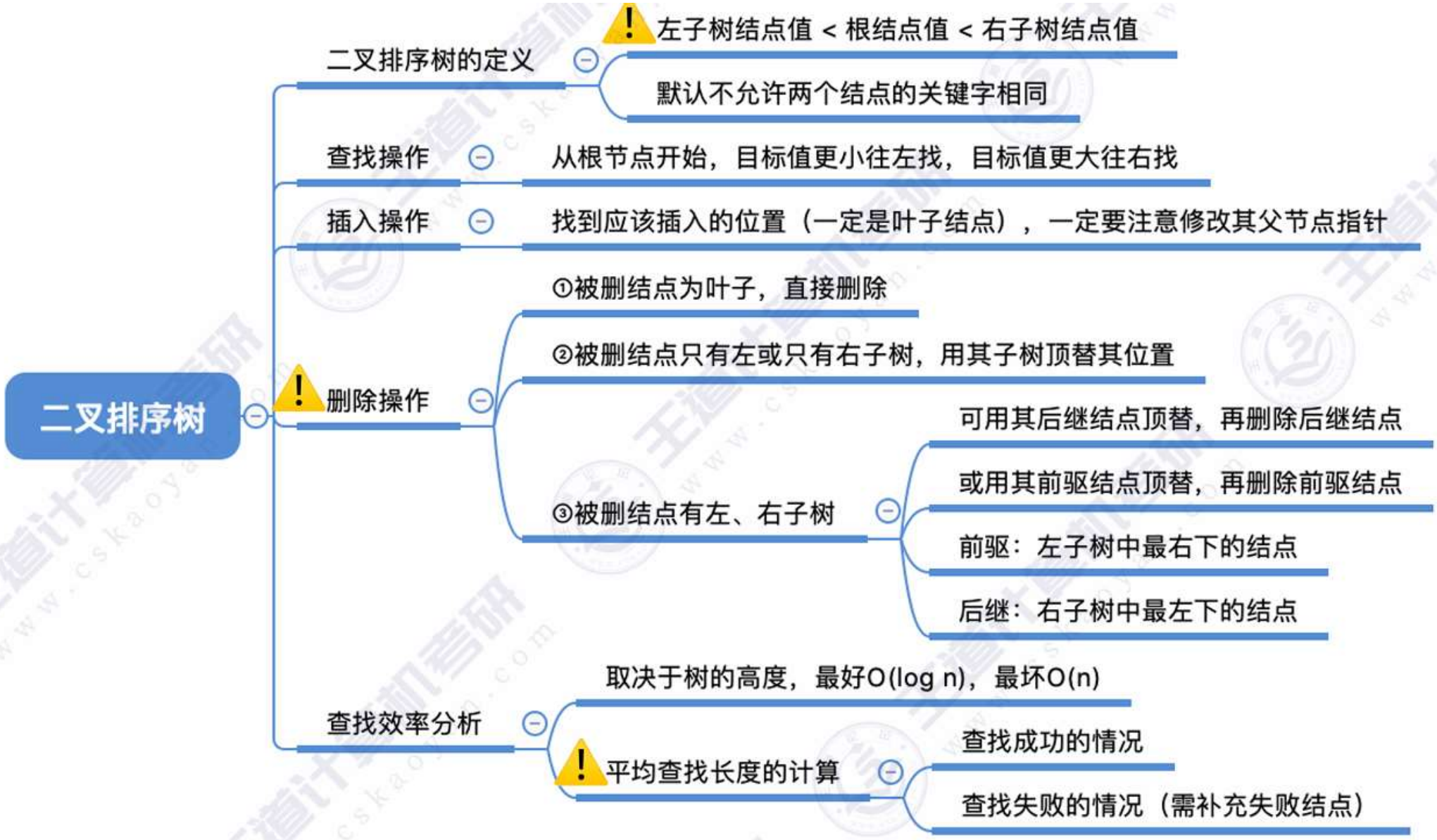
> **特征：** 中序遍历 BST 得到一个严格递增的有序序列。

插入： 总是作为新叶子节点插入。

删除： 若被删节点有两个孩子，用其中序前驱或中序后继替代。

BST 查找效率取决于**树的形态**：

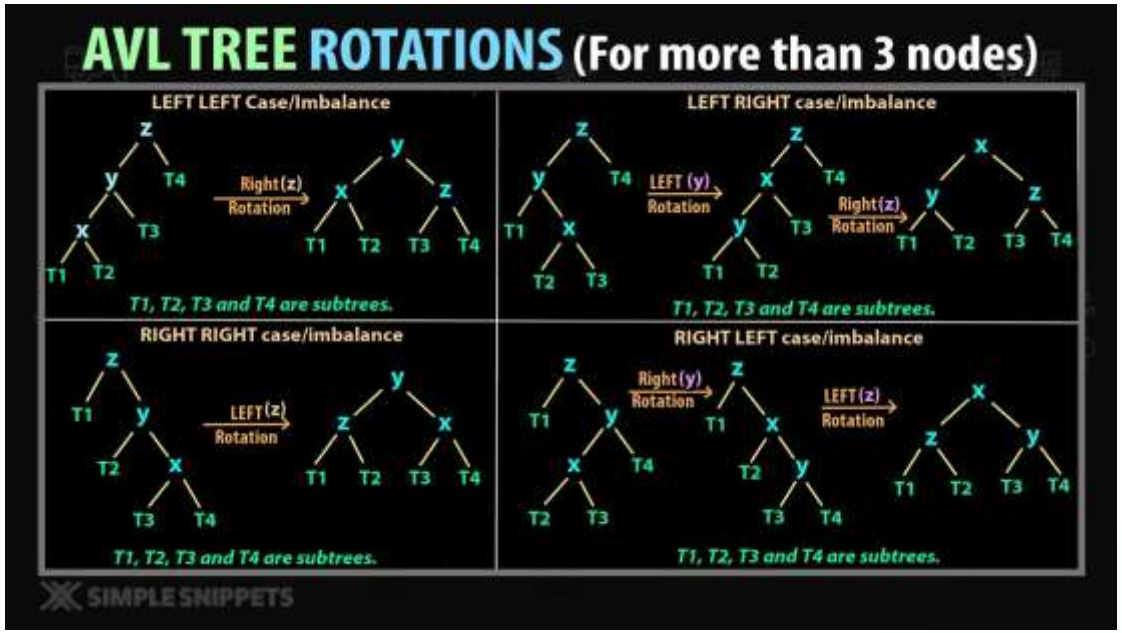
- 最好：$O(\log_2 n)$ (平衡)
- 最坏：O(n) (单支树)


二叉排序树
二叉排序树的定义
左子树结点值 < 根结点值 < 右子树结点值
默认不允许两个结点的关键字相同
查找操作
从根节点开始，目标值更小往左找，目标值更大往右找
插入操作
找到应该插入的位置（一定是叶子结点），一定要注意修改其父节点指针
删除操作
①被删结点为叶子，直接删除
②被删结点只有左或只有右子树，用其子树顶替其位置
③被删结点有左、右子树
可用其后继结点顶替，再删除后继结点
或用其前驱结点顶替，再删除前驱结点
前驱：左子树中最右下的结点
后继：右子树中最左下的结点
查找效率分析
取决于树的高度，最好O(log n)，最坏O(n)
平均查找长度的计算
查找成功的情况
查找失败的情况（需补充失败结点）

平衡二叉树 (AVL Tree)

定义： BST 的任一节点的左右子树高度差绝对值 ≤ 1。

$$\text{平衡因子 (BF)} = h_L - h_R$$



AVL 树的四种平衡调整

类型	插入位置	调整方法
LL 型	左孩子的左子树	右单旋
RR 型	右孩子的右子树	左单旋
LR 型	左孩子的右子树	先左旋后右旋
RL 型	右孩子的左子树	先右旋后左旋

🚨 **核心：** 旋转的目的是降低子树高度，维持平衡因子在 $\{-1, 0, 1\}$ 。

知识回顾与重要考点

- 平衡二叉树
  - 定义
    - 树上任一结点的左子树和右子树的高度之差不超过1
    - 结点的平衡因子=左子树高-右子树高
  - 插入操作
    - 和二叉排序树一样，找合适的位置插入
    - 新插入的结点可能导致其祖先们平衡因子改变，导致失衡
  - ! 调整“不平衡”
    - 找到最小不平衡子树进行调整，记最小不平衡子树的根为A
    - LL：在A的左孩子的左子树插入导致A不平衡，将A的左孩子右上旋
    - RR：在A的右孩子的右子树插入导致A不平衡，将A的右孩子左上旋
    - LR：在A的左孩子的右子树插入导致A不平衡，将A的左孩子的右孩子 先左上旋再右上旋
    - RL：在A的右孩子的左子树插入导致A不平衡，将A的右孩子的左孩子 先右上旋再左上旋
  - 查找效率分析
    - 考点：高为h的平衡二叉树最少有几个结点——递推求解
    - 平衡二叉树最大深度为 O(log n)，平均查找长度/查找的时间复杂度为 O(log n)

! 实现 f 向右下旋转，p 向右上旋转：
其中 f是爹，p为左孩子，gf为f他爹
① f->lchild = p->rchild;
② p->rchild = f;
③ gf->lchild/rchild = p;

! 实现 f 向左下旋转，p 向左上旋转：
① f->rchild = p->lchild;
② p->lchild = f;
③ gf->lchild/rchild = p;

平衡二叉树的**删除**操作具体步骤：

①删除结点（方法同“二叉排序树”）
- 若删除的结点是叶子，直接删。
- 若删除的结点只有一个子树，用子树顶替删除位置
- 若删除的结点有两棵子树，用前驱（或后继）结点顶替，并转换为对前驱（或后继）结点的删除。

②一路向北找到最小不平衡子树，找不到就完结撒花

③找最小不平衡子树下，“个头”最高的儿子、孙子

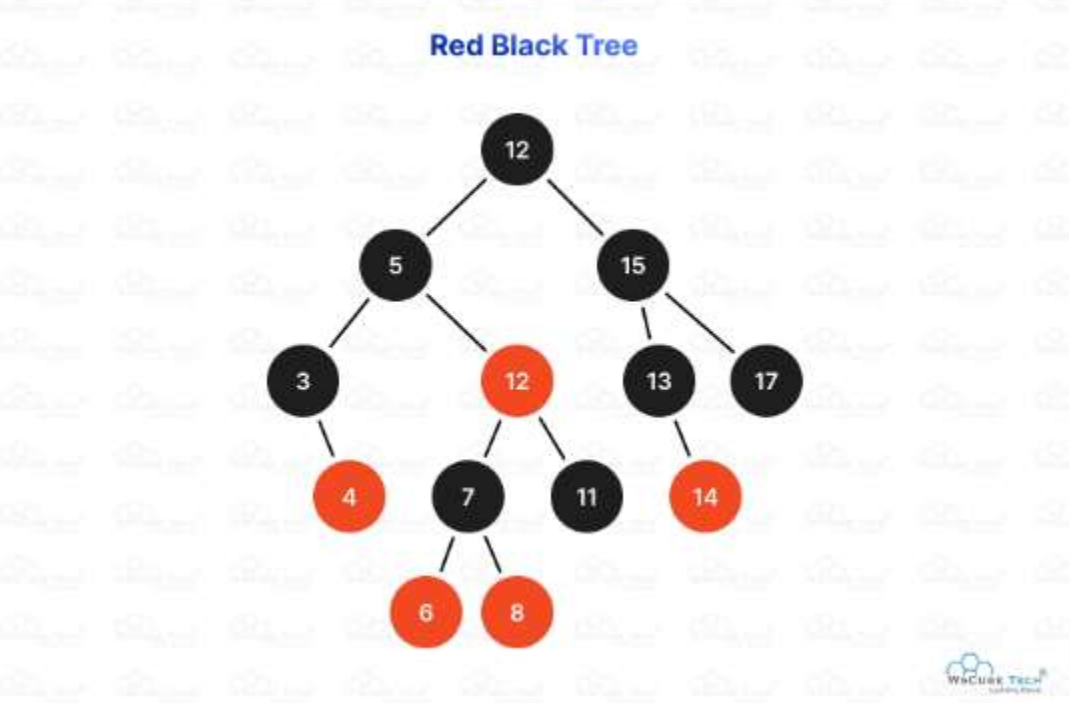
④根据孙子的位置，调整平衡（LL/RR/LR/RL）
- 孙子在LL：儿子右单旋
- 孙子在RR：儿子左单旋
- 孙子在LR：孙子先左旋，再右旋
- 孙子在RL：孙子先右旋，再左旋

平衡二叉树删除操作时间复杂度=$O(\log_2 n)$

⑤如果不平衡向上传导，继续②
- 对最小不平衡子树的旋转可能导致树变矮，从而导致上层祖先不平衡（不平衡向上传递）

红黑树的 5 大性质

1. 每个节点或是红的，或是黑的。
2. **根节点是黑的。**
3. 叶节点 (NIL) 是黑的。
4. 红色节点的两个孩子必是黑的 (不连续红)。
5. **黑路同：** 任一节点到叶子的路径包含相同数量的黑节点。




红黑树RBT
相关概念
黑高 —— 一个结点的“黑高”：从某结点出发（不含该结点）到达任一空叶结点的路径上黑结点总数
叶结点 —— 在RBT中，“叶结点”通常指“失败结点”（又名：空叶结点/NULL结点/外部结点）
定义
左根右 —— RBT是一种BST，需满足 左≤根≤右
根叶黑 —— 根节点、叶结点一定是黑色
不红红 —— 任何一条查找路径上不能连续出现两个红结点
黑路同 —— 从任一结点出发，到达任一空叶结点的路径上经过的黑结点数量都相同
性质
从根节点到叶结点的最长路径不大于最短路径的2倍
有n个内部节点的红黑树高度 h≤2log (n+1)
插入
新结点为根染黑
新结点非根染红，若插入后违反“不红红”，则看叔叔的脸色
黑叔：旋转+染色
LL —— 右单旋，父换爷+染色
RR —— 左单旋，父换爷+染色
LR —— 左、右双旋，儿换爷+染色
RL —— 右、左双旋，儿换爷+染色
红叔：染色+变新 —— 叔父爷染色，爷变为新结点

红黑树插入 —— 总结

红黑树 插入

左根右
根叶黑
不红红
黑路同

插入结点默认是红色结点

如果插入后性质被破坏，则根据下面三种情况做调整：

插入结点是根结点 → 直接变黑

破坏了根叶黑

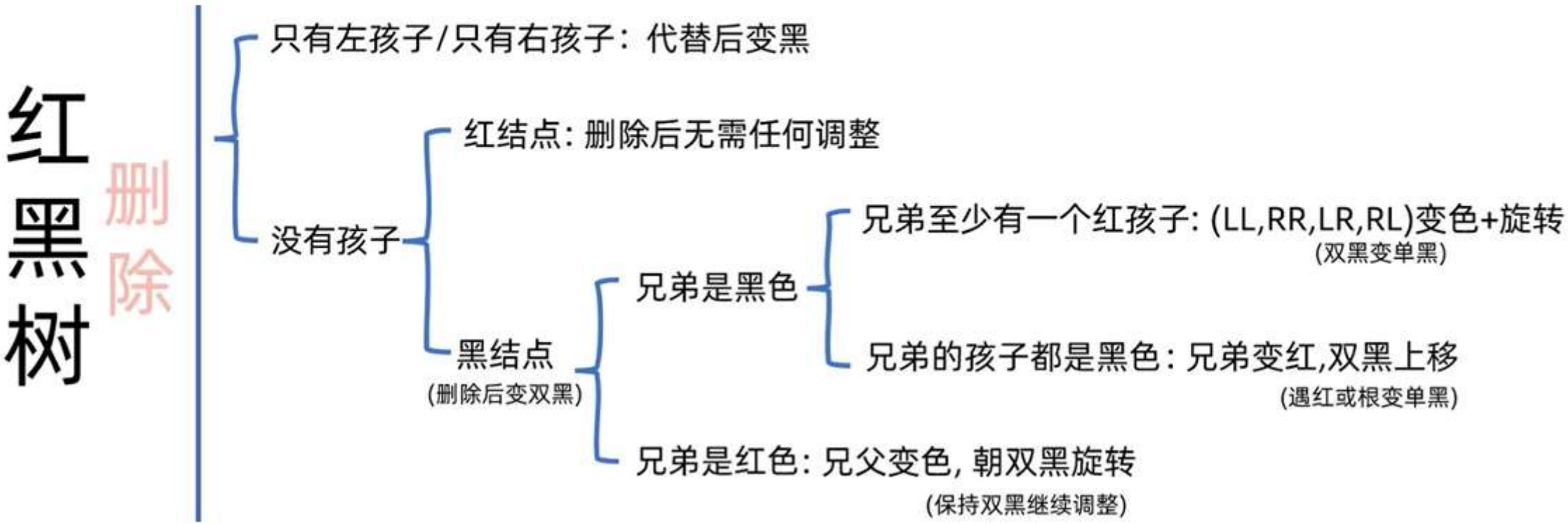
插入结点的叔叔是红色 → 叔父爷变色，爷爷变插入结点

插入结点的叔叔是黑色 → (LL,RR,LR,RL)旋转，然后变色

破坏了不红红

[红黑树 - 定义, 插入, 构建_哔哩哔哩_bilibili]

总结：红黑树删除



[红黑树 - 删除_哔哩哔哩_bilibili]

[Red/Black Tree Visualization]

红黑树 vs 平衡二叉树 (AVL)

维度	AVL 树	红黑树 (RBT)
平衡标准	高度差 ≤ 1 (更严格)	黑高度相同 (更宽松)
查找性能	略优	接近
插入/删除	旋转频繁	旋转较少
适用场景	读多写少	读写平衡 (如 Java TreeMap)

🚨 **结论：** RBT 的高度不大于 $2\log_2(n+1)$，查找复杂度仍为 $O(\log n)$。

B 树 (多路平衡搜索树)

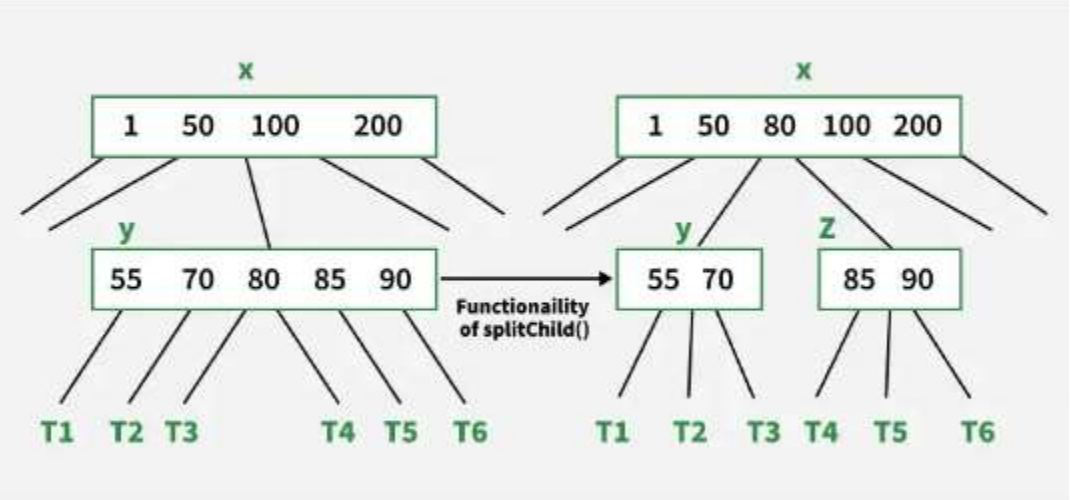
定义： m 阶 B 树中，根节点孩子数 [2, m]，非根非叶节点孩子数 $[\lceil m/2 \rceil, m]$。

- 所有叶子节点都在同一层 (代表查找失败)。
- 关键字个数范围： $[\lceil m/2 \rceil - 1, m - 1]$。
- **目的：** 减少磁盘 I/O 次数。

B 树的插入与分裂

步骤：

1. 找到插入叶子位置。
2. 若关键字超限 (> m-1)，进行分裂。
3. 将中间位置 ($\lceil m/2 \rceil$)关键字向上层提拔。
4. 提拔可能导致父节点连锁分裂。



B+ 树：数据库索引的首选

特征 A

非叶节点仅做索引，不存数据。数据全在叶子节点。

特征 B

叶子节点包含全部关键字，且按序链成双向链表。

特征 C

支持随机查找和顺序查找。

🚨 **区别：** B+ 树分支节点关键字个数与孩子数相同，B 树则比孩子数少 1。

	m阶B树	m阶B+树
类比	二叉查找树的进化——>m叉查找树	分块查找的进化——>多级分块查找
关键字与分叉	n个关键字对应n+1个分叉（子树）	n个关键字对应n个分叉
结点包含的信息	所有结点中都包含记录的信息	只有最下层叶子结点才包含记录的信息 (可使树更矮)
查找方式	不支持顺序查找。查找成功时，可能停在任何一层结点，查找速度“不稳定”	支持顺序查找。查找成功或失败都会到达最下一层结点，查找速度“稳定”
相同点	除根节点外，最少 $\lceil m/2 \rceil$ 个分叉（确保结点不要太“空”） 任何一个结点的子树都要一样高（确保“绝对平衡”）	

散列表基本概念

基本思想： 通过散列函数 H(key) 将关键字直接映射到存储地址。

冲突 (Collision)： 不同关键字映射到同一地址。

常用构造方法：除留余数法

$$H(key) = key \% p$$

其中 p 通常为不大于表长 m 的最大质数。

冲突处理：开放定址法

方法	探测增量 d_i	备注
线性探测法	1, 2, ..., m-1	容易产生“堆积”现象
平方探测法	$1^2, -1^2, 2^2, -2^2, \ldots$	有效避免堆积
伪随机序列法	随机数序列	-

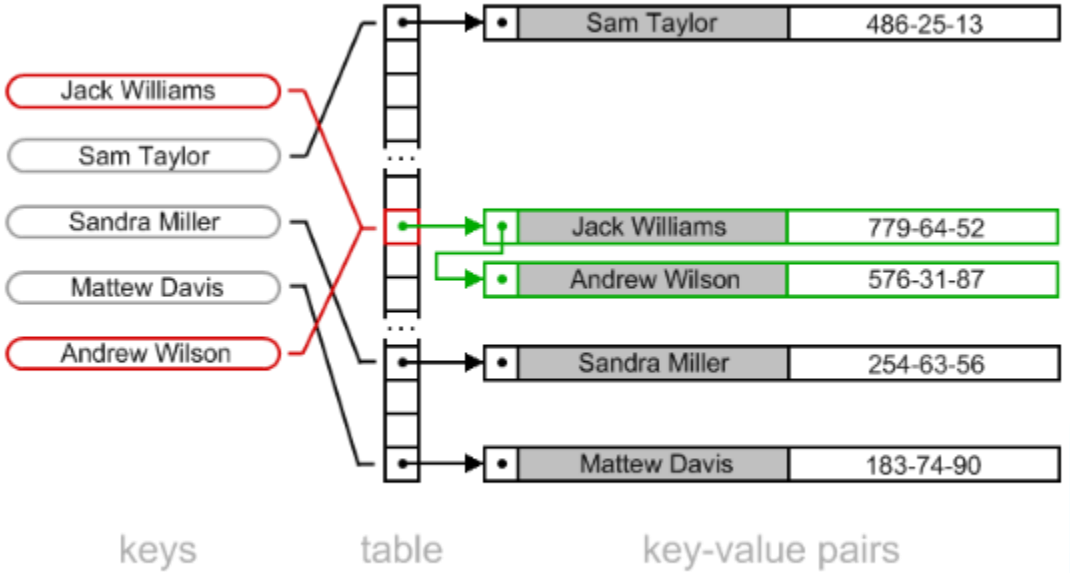
🚨 **注意：** 线性探测法删除节点不能物理清除，需设置删除标记。

冲突处理：拉链法 (Chaining)

逻辑： 将所有散列地址相同的记录存储在同一个单链表中。

优点：

- 不产生堆积。
- 删除节点方便。
- 适合无法确定表长的情况。



散列表查找效率 (ASL) 计算

散列查找效率取决于：散列函数、冲突处理方法、**装填因子** (α)。

$$\alpha = \frac{\text{表中记录数 n}}{\text{散列表长度 m}}$$

🚨 **失败 ASL 计算陷阱：** 分母是散列函数 H(key) 的范围 p，而不是表长 m 或元素个数 n。

朴素模式匹配 (BF 算法)

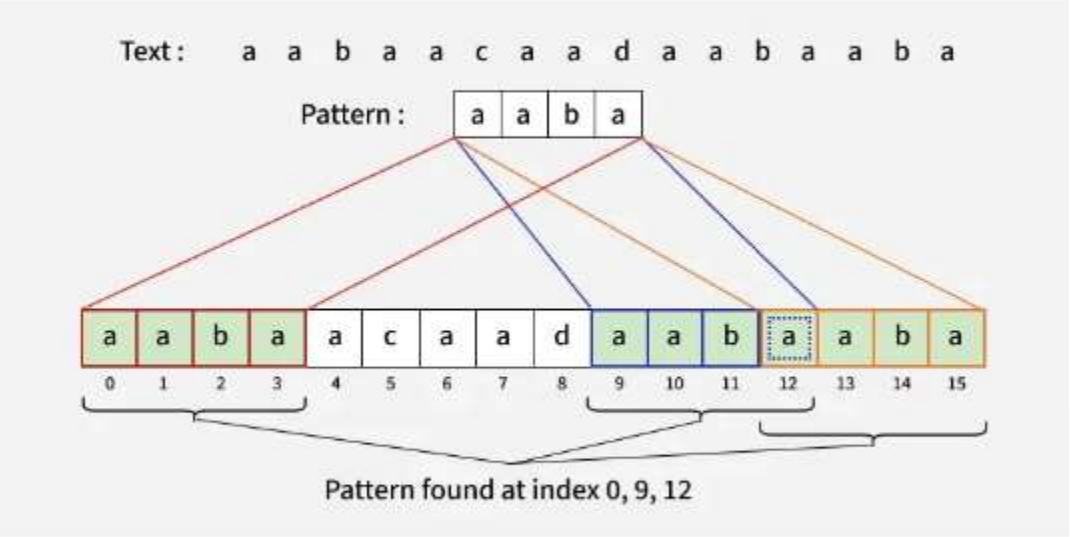
逻辑： 暴力枚举。主串指针 i 与模式串指针 j 一一比对，失败则 i 回溯到 i-j+1。

- 时间复杂度：$O(n \times m)$(最坏)。
- 缺点：主串指针大量重复回溯。

KMP 算法核心思想

智慧： 利用已经匹配过的部分信息，**主串指针 i 不回溯**，仅移动模式串指针 j 到 next[j]。

> 复杂度：O(n + m)



Next 数组计算规则

定义： next[j] 表示在模式串第 j 个字符匹配失败时，应跳转到的新指针位置。

> **手算步骤：**
>
> 1. next[1] = 0, next[2] = 1。
> 2. 对于第 j 位，寻找最长相等的前后缀长度 k。
> 3. next[j] = k + 1。

🚨 **优化：** nextval 数组解决如 aaaaab 这种极端重复串的无效比对问题。

KMP算法

根据模式串T，求出 next 数组 → 利用next数组进行匹配（主串指针不回溯）

KMP算法精髓：利用好已经匹配过的模式串的信息

next数组:

next[0]	next[1]	next[2]	next[3]	next[4]	next[5]	next[6]
	0	1	1	2	2	3

```
if (S[i] !=T[j])    j=next[j];

if (j==0)    { i++; j++ }
```

KMP算法，最坏时间复杂度 O(m+n)

其中，求 next 数组时间复杂度 O(m)
模式匹配过程最坏时间复杂度 O(n)

```
int Index_KMP(SString S,SString T,int next[]){
    int i=1, j=1;
    while(i<=S.length&&j<=T.length){
        if(j==0||S.ch[i]==T.ch[j]){
            ++i;
            ++j;                    //继续比较后继字符
        }
        else
            j=next[j];              //模式串向右移动
    }
    if(j>T.length)
        return i-T.length;          //匹配成功
    else
        return 0;
}
```

KMP算法——求next数组

根据模式串T，求出 next 数组

T = ‘abaabc’

next数组:

next[0]	next[1]	next[2]	next[3]	next[4]	next[5]	next[6]
	0	1	1	2	2	3

```
if (S[i] !=T[j])     j=next[j];

if (j==0)     { i++; j++ }
```

next[1]都无脑写 0
next[2]都无脑写 1

其他 next：在不匹配的位置前，划一根美丽的分界线
模式串一步一步往后退，直到分界线之前“能对上”，或模式串完全跨过分界线为止。此时 j 指向哪儿，next数组值就是多少

KMP算法，最坏时间复杂度 O(m+n)

其中，求 next 数组时间复杂度 O(m)
模式匹配过程最坏时间复杂度 O(n)

KMP算法的进一步优化

根据模式串T，求出 next 数组

利用next数组进行匹配（主串指针不回溯）

T = 'abaabc'

next数组:

next[0]	next[1]	next[2]	next[3]	next[4]	next[5]	next[6]
	0	1	1	2	2	3

优化

nextval数组:

nextval[0]	nextval[1]	nextval[2]	nextval[3]	nextval[4]	nextval[5]	nextval[6]
	0	1	0	2	1	3

使用nextval数组

```
int Index_KMP(SString S,SString T,int next[]){
    int i=1, j=1;
    while(i<=S.length&&j<=T.length){
        if(j==0||S.ch[i]==T.ch[j]){
            ++i;
            ++j;                    //继续比较后继字符
        }
        else
            j=next[j];              //模式串向右移动
    }
    if(j>T.length)
        return i-T.length;          //匹配成功
    else
        return 0;
}
```

各查找算法复杂度横向对比

算法	平均时间复杂度
顺序查找	$O(n)$
折半查找	$O(\log n)$
分块查找	$O(\sqrt{n})$
树形查找	$O(\log n)$
散列查找	$O(1)$

Algorithm	Best Time Complexity	Average Time Complexity	Worst Time Complexity	Worst Space Complexity
Linear Search	O(1)	O(n)	O(n)	O(1)
Binary Search	O(1)	O(log n)	O(log n)	O(1)
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Merge Sort	O(nlogn)	O(nlogn)	O(nlogn)	O(n)
Quick Sort	O(nlogn)	O(nlogn)	O(n^2)	O(log n)
Heap Sort	O(nlogn)	O(nlogn)	O(nlogn)	O(n)
Bucket Sort	O(n+k)	O(n+k)	O(n^2)	O(n)
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)
Tim Sort	O(n)	O(nlogn)	O(nlogn)	O(n)
Shell Sort	O(n)	O((nlog(n))^2)	O((nlog(n))^2)	O(1)

如何选择合适的查找算法？



静态表

数据不常变动，选**折半查找**。



动态表

频繁插入删除，选**平衡树 (AVL/RBT)**。



超极速查找

忽略空间，追求 O(1)，选**散列表**。

典型真题解析：折半查找 ASL

> **题目：** 对有序表 (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11) 进行折半查找，成功 ASL 为多少?

解析：

1. 画判定树。1层：1个；2层：2个；3层：4个；4层：4个。
2. 计算：$(1 \times 1 + 2 \times 2 + 4 \times 3 + 4 \times 4)/11 = 33/11 = 3$。

典型真题解析：Hash 失败 ASL

> **题目：** H(key) = $key\%7$，用线性探测法处理冲突，查找到空地址才算失败。p=7，失败 ASL 的分母应是多少？

解析：

分母固定为 p=7。因为 H(key) 产生的地址只可能在 $0 \sim 6$ 之间，只需要计算这 7 个起始地址查找失败所需的平均步数。

KMP Next 数组计算示例

> 模式串：ababaa

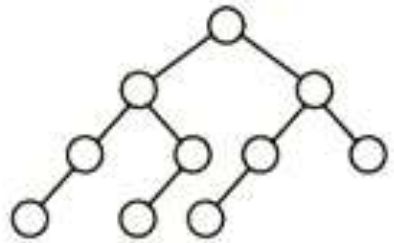
手算：

- j=1: `a` → 0
- j=2: `ab` → 1
- j=3: `aba` $(a = a) \rightarrow 1 + 1 = 2$
- j=4: `abab` $(ab = ab) \rightarrow 2 + 1 = 3$
- j=5: `ababa` $(aba = aba) \rightarrow 3 + 1 = 4$
- j=6: `ababaa` $(a = a) \rightarrow 1 + 1 = 2$

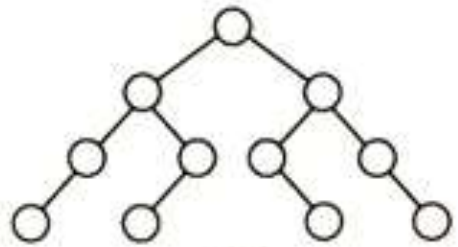
结果串：(0, 1, 2, 3, 4, 2)

23. 【2017 统考真题】下列二叉树中，可能成为折半查找判定树（不含外部结点）的是（　）。

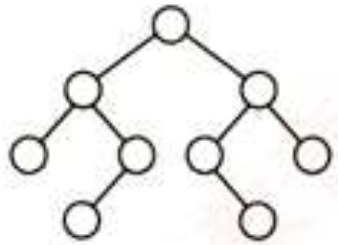
A.



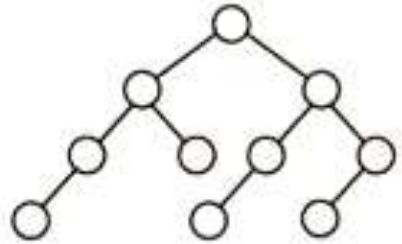
B.



C.



D.



23. A

对于给定的一个有序查找表，其对应的折半查找判定树是确定且唯一的。在折半查找算法中，mid=⌊(low+high)/2⌋，因此若表中初始有 $2n+1$ 个元素，则 mid 分割后，左右子树各有 n 个元素；若表中初始有 $2n$ 个元素，则 mid 分割后，左子树有 $n-1$ 个元素，右子树有 n 个元素。即左子树的元素个数或者与右子树的元素个数相等，或者比右子树少一个。若令 mid=⌈(low+high)/2⌉，不难理解，左子树的元素个数或者与右子树的元素个数相等，或者比右子树多一个。对于选项 A，树中每个左子树都与右子树的结点个数相等，或者多一个结点，符合向上取整的规则。对于选项 B、C、D，存在有的左子树比右子树多一个结点，有的左子树比右子树少一个结点，不符合折半查找的规则。

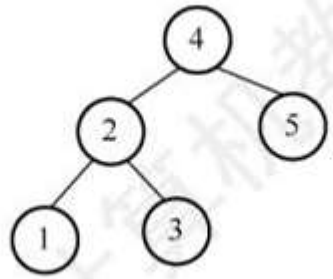
24. 【2023 统考真题】对含 600 个元素的有序顺序表进行折半查找，关键字间的比较次数最多是（ ）。

A. 9　　B. 10　　C. 30　　D. 300

24. B

用折半查找法查找给定值的比较次数最多不超过折半查找判定树的高度。折半查找判定树的树高 $h=\lceil \log_2(n+1) \rceil$，将 $n=600$ 代入，结果为 10。

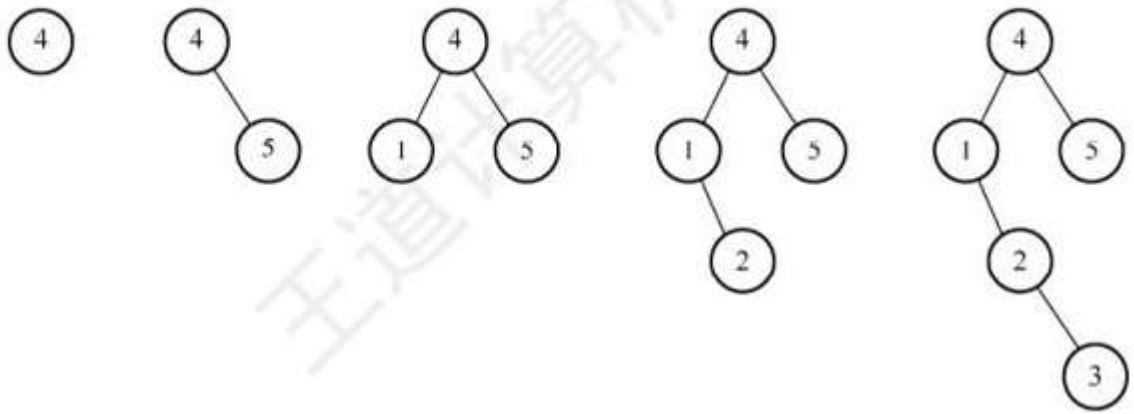
32. 【2020 统考真题】下列给定的关键字输入序列中，不能生成右边二叉排序树的是（ ）。



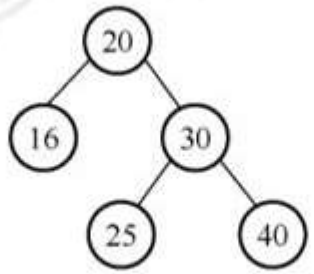
A. 4, 5, 2, 1, 3　　B. 4, 5, 1, 2, 3　　C. 4, 2, 5, 3, 1　　D. 4, 2, 1, 3, 5

32. B

每个选项都逐一验证，选项 B 生成二叉排序树的过程如下图所示，显然错误。



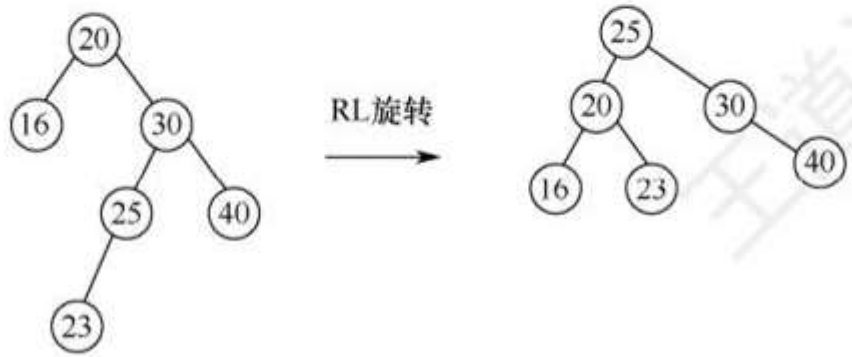
33.【2021 统考真题】给定平衡二叉树如下图所示，插入关键字 23 后，根中的关键字是（ ）。



A. 16　　B. 20　　C. 23　　D. 25

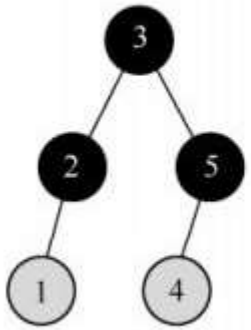
33. D

关键字 23 的插入位置为 25 的左孩子，此时破坏了平衡的性质，需要对平衡二叉树进行调整。最小不平衡子树就是该树本身，插入位置在根结点的右子树的左子树上，因此需要进行 RL 旋转，RL 旋转过程如下图所示，旋转完成后根结点的关键字为 25，所以选 D。

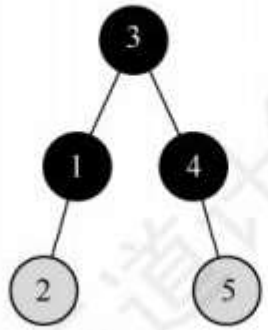


19. 下列四个选项中，满足红黑树定义的是（ ）。

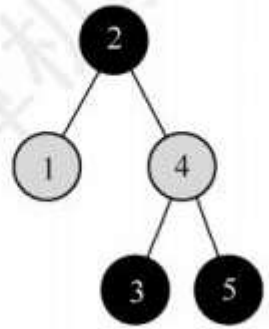
A.



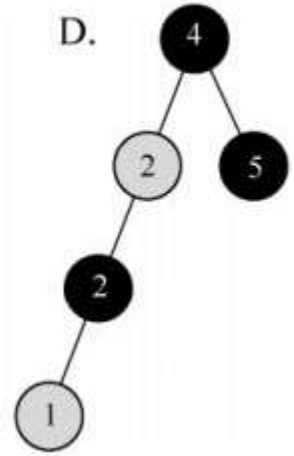
B.



C.



D.



19. A

红黑树是一种特殊的二叉排序树，B 项不满足二叉排序树的性质。C 项中，结点 2 的左右黑结点数不同。D 项中，结点 3 的左右黑结点数不同。只有 A 项满足红黑树的定义。

插入结点默认是红色结点

如果插入后性质被破坏, 则根据下面三种情况做调整:

插入结点是根结点 → 直接变黑

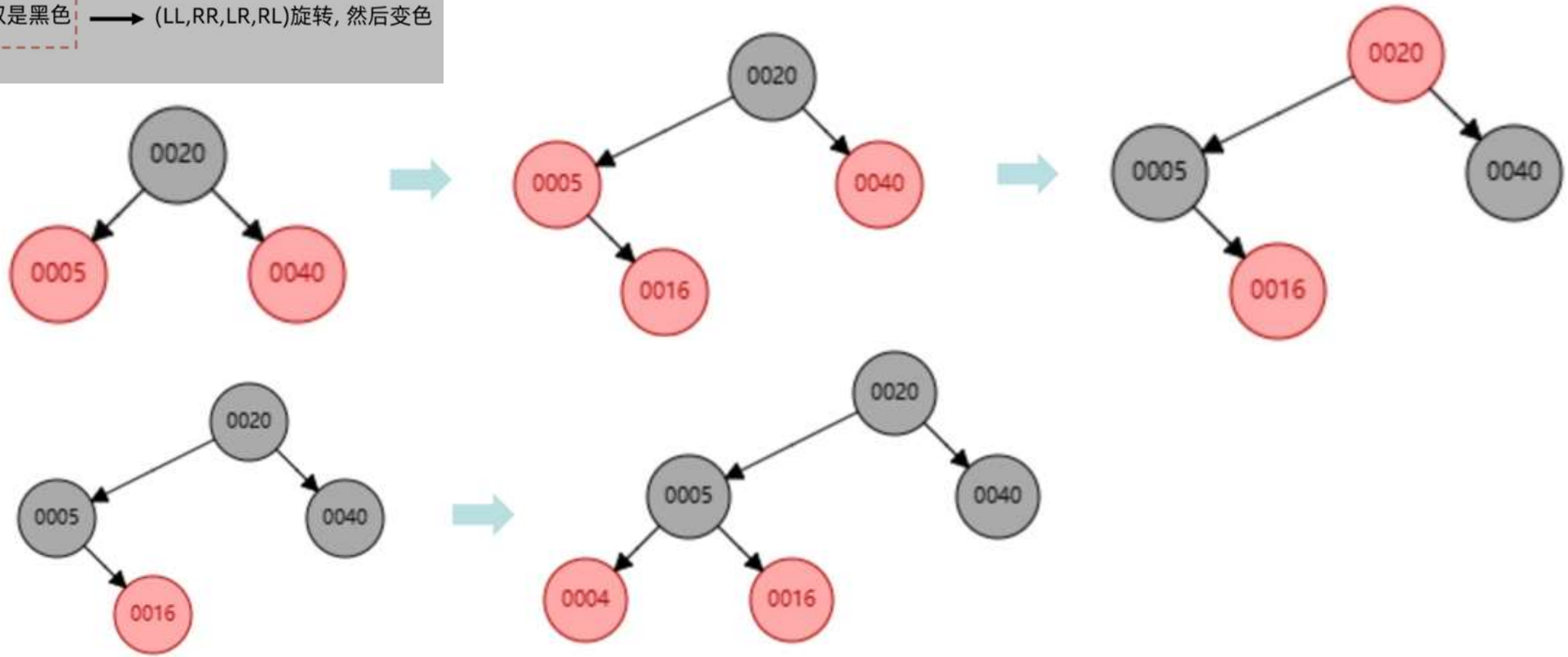
破坏了根叶黑

插入结点的叔叔是红色 → 叔父爷变色, 爷爷变插入结点

插入结点的叔叔是黑色 → (LL,RR,LR,RL)旋转, 然后变色

破坏了不红红

按序插入[20, 5, 40, 16, 4,3,2,1]



插入结点默认是红色结点

如果插入后性质被破坏，则根据下面三种情况做调整：

插入结点是根结点 → 直接变黑

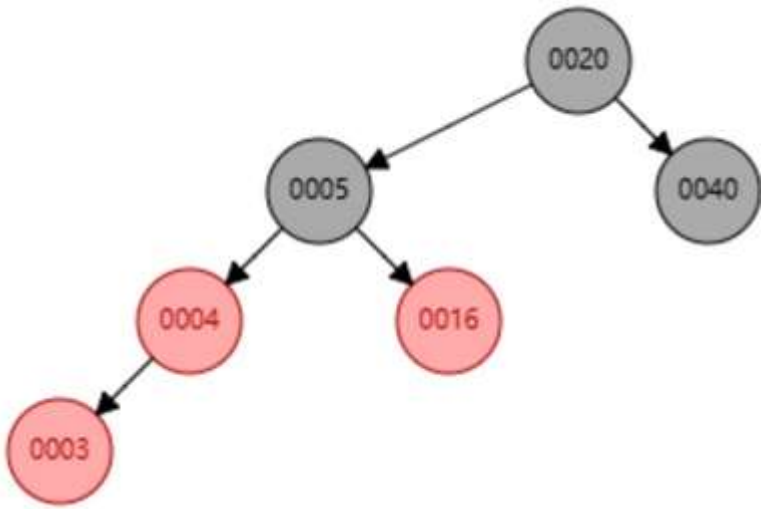
破坏了根叶黑

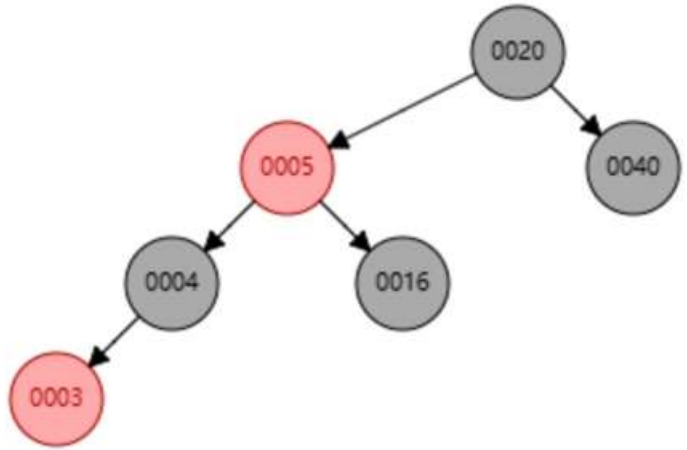
插入结点的叔叔是红色 → 叔父爷变色，爷爷变插入结点

插入结点的叔叔是黑色 → (LL,RR,LR,RL)旋转，然后变色

破坏了不红红

按序插入[20, 5, 40, 16, 4,3,2,1]





简单的改变颜色，未发生旋转

BF=2（与AVL不同的地方，并没有那么严格）

插入结点默认是红色结点

如果插入后性质被破坏, 则根据下面三种情况做调整:

插入结点是根结点 → 直接变黑

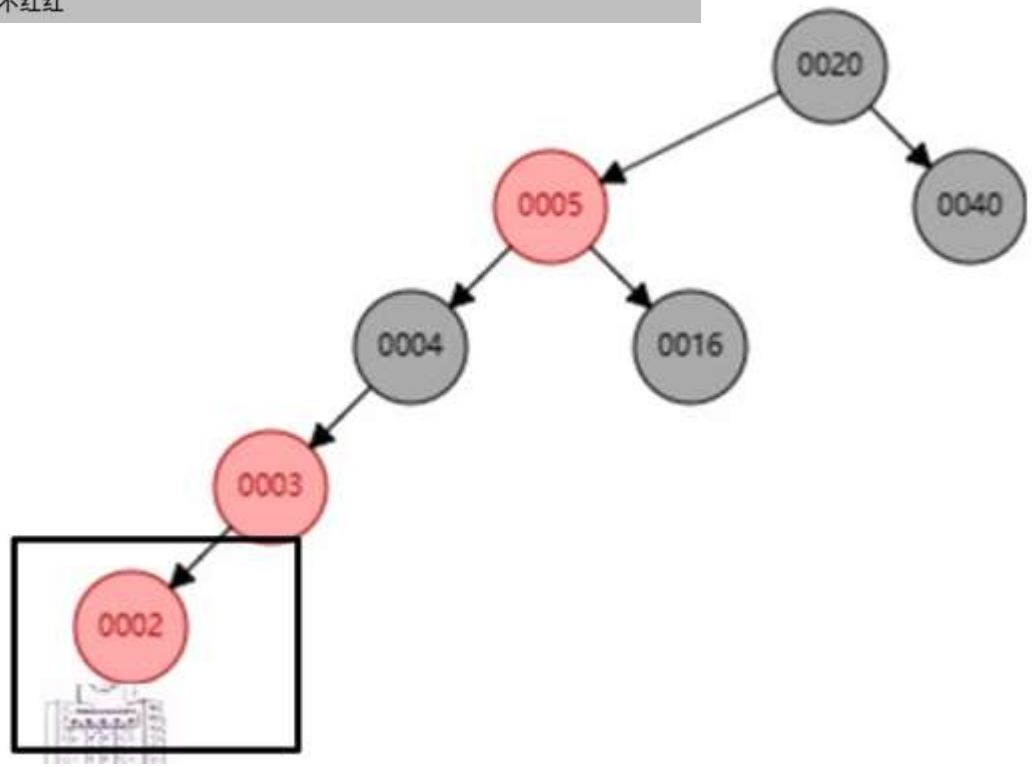
破坏了根叶黑

插入结点的叔叔是红色 → 叔父爷变色, 爷爷变插入结点

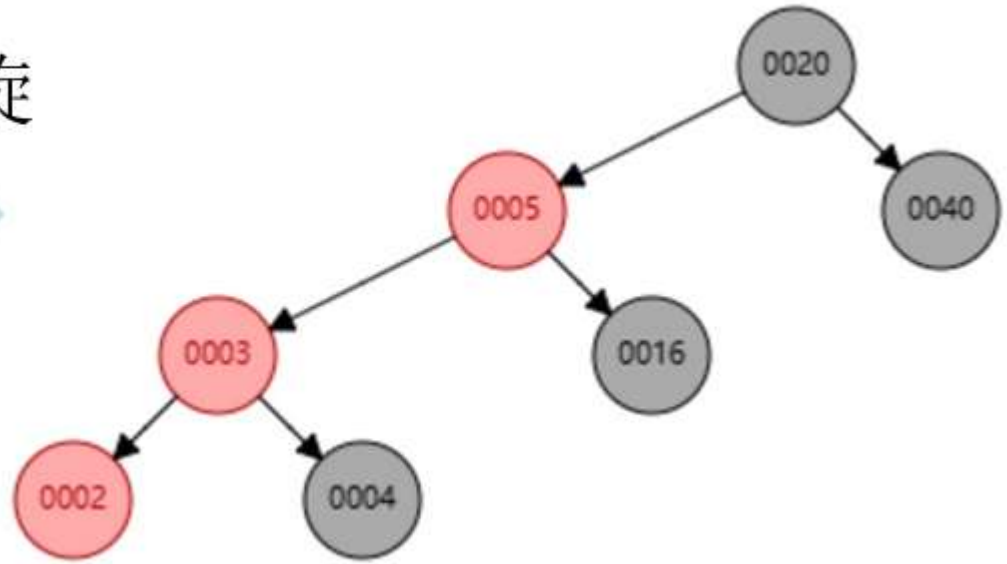
插入结点的叔叔是黑色 → (LL,RR,LR,RL)旋转, 然后变色

破坏了不红红

按序插入[20, 5, 40, 16, 4,3,2,1]



右旋



插入结点默认是红色结点

如果插入后性质被破坏, 则根据下面三种情况做调整:

插入结点是根结点 → 直接变黑

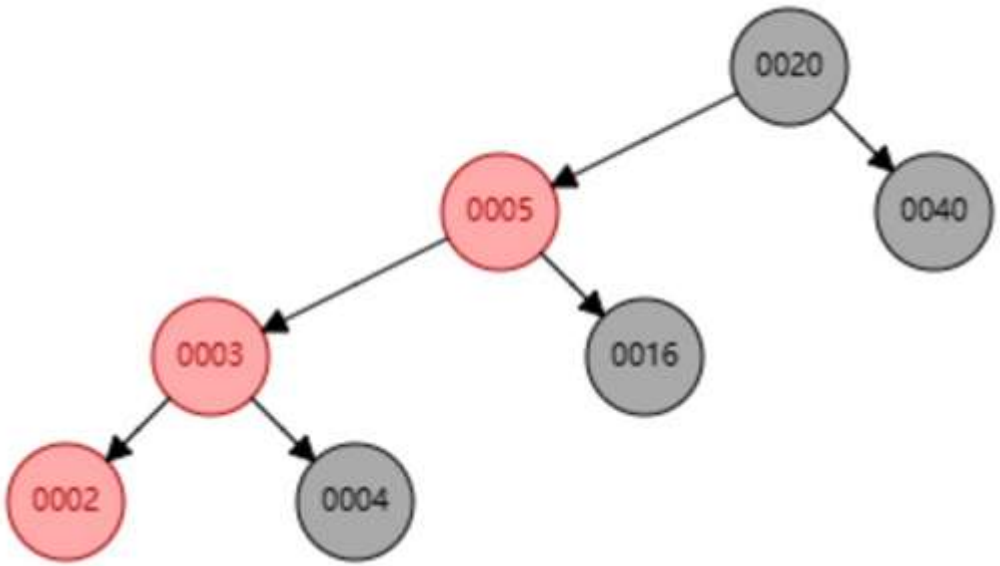
破坏了根叶黑

插入结点的叔叔是红色 → 叔父爷变色, 爷爷变插入结点

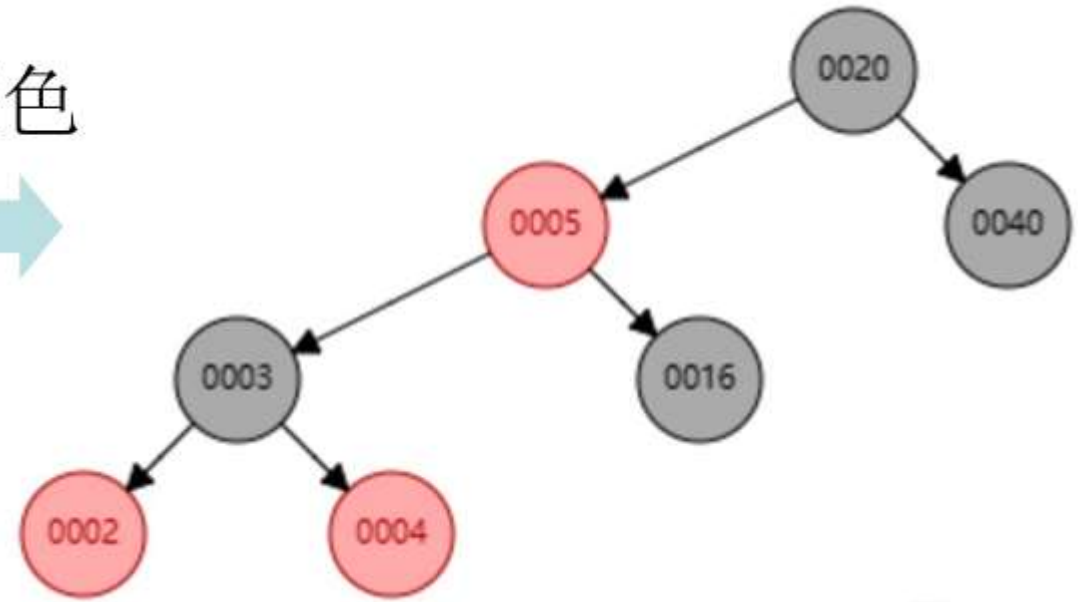
插入结点的叔叔是黑色 → (LL,RR,LR,RL)旋转, 然后变色

破坏了不红红

按序插入[20, 5, 40, 16, 4,3,2,1]



变色



插入结点默认是红色结点

如果插入后性质被破坏, 则根据下面三种情况做调整:

插入结点是根结点 → 直接变黑

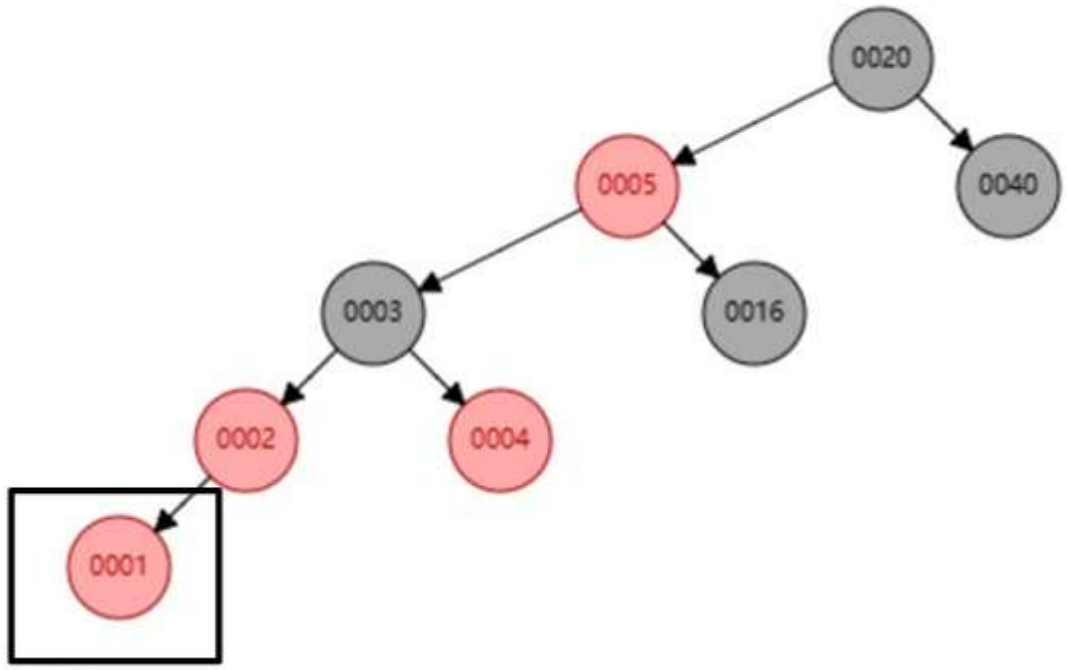
破坏了根叶黑

插入结点的叔叔是红色 → 叔父爷变色, 爷爷变插入结点

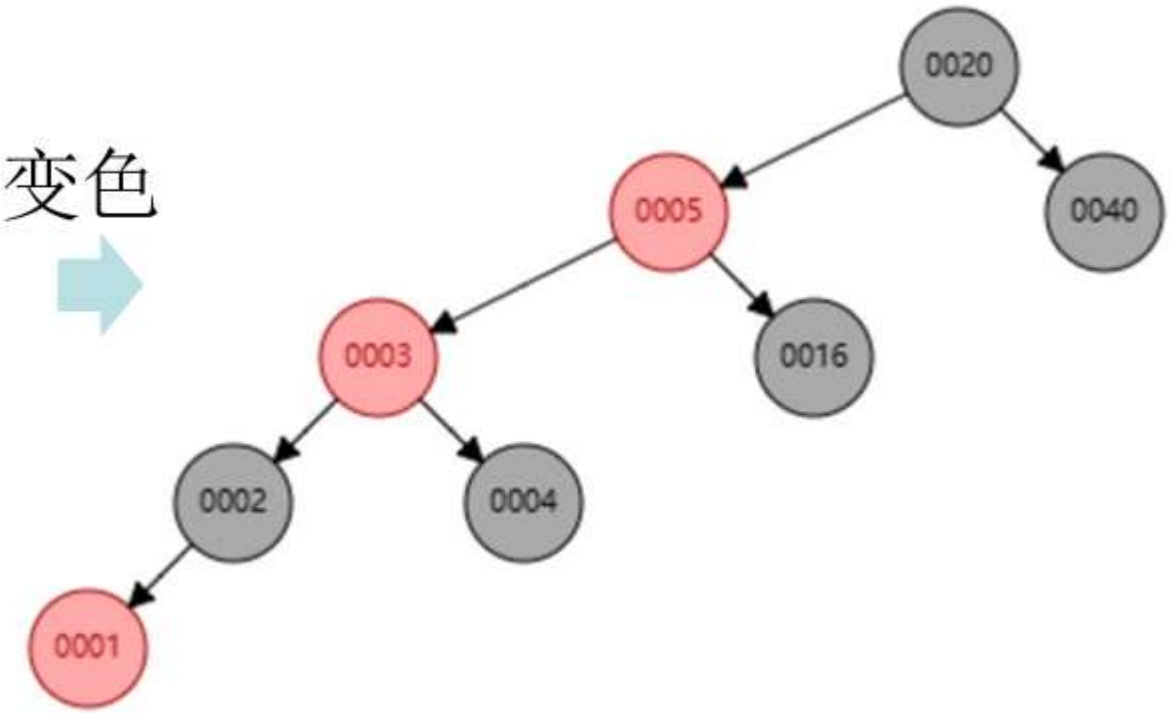
插入结点的叔叔是黑色 → (LL,RR,LR,RL)旋转, 然后变色

破坏了不红红

按序插入[20, 5, 40, 16, 4,3,2,1]



变色



递归调整

插入结点默认是红色结点

如果插入后性质被破坏, 则根据下面三种情况做调整:

插入结点是根结点 → 直接变黑

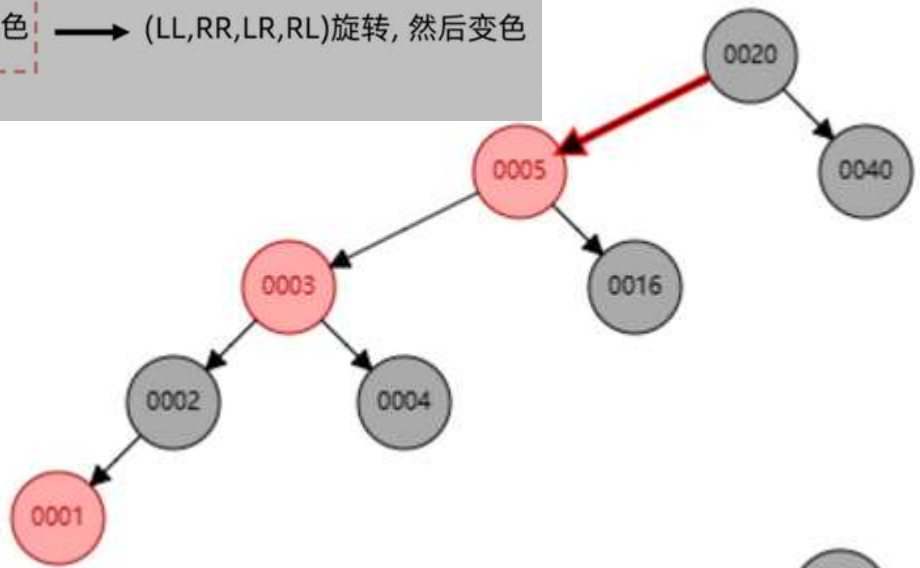
破坏了根叶黑

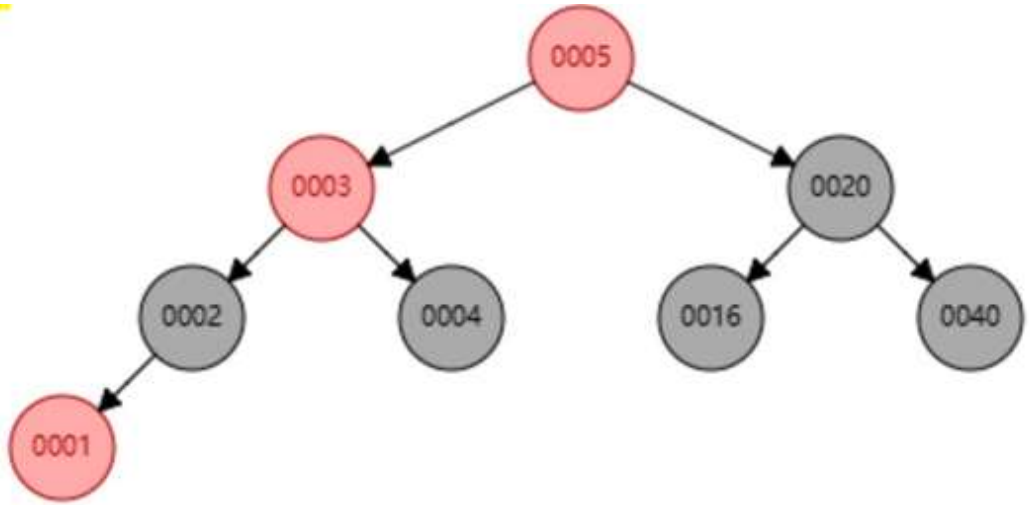
插入结点的叔叔是红色 → 叔父爷变色, 爷爷变插入结点

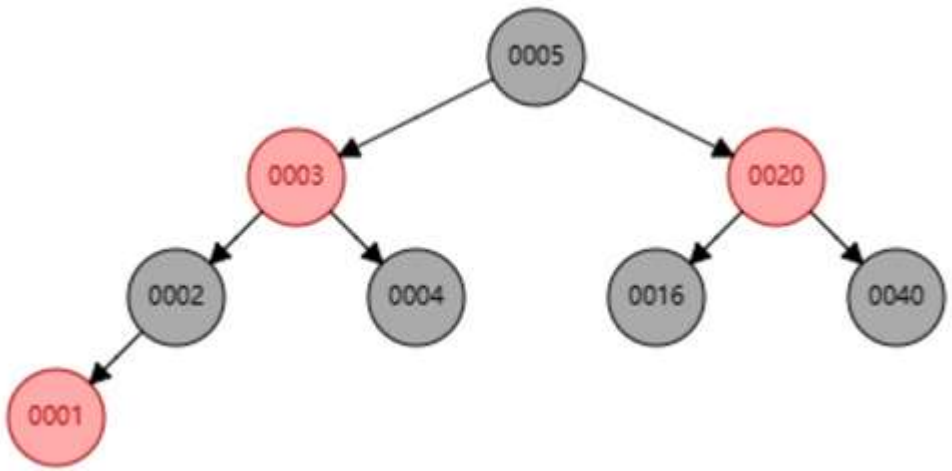
插入结点的叔叔是黑色 → (LL,RR,LR,RL)旋转, 然后变色

破坏了不红红

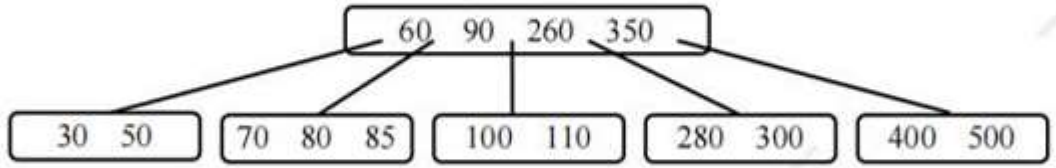
按序插入[20, 5, 40, 16, 4,3,2,1]







21. 【2022 统考真题】在下图所示的 5 阶 B 树 T 中，删除关键字 260 之后需要进行必要的调整，得到新的 B 树 T_1。下列选项中，不可能是 T_1 根结点中关键字序列的是（ ）。



A. 60, 90, 280　　B. 60, 90, 350

C. 60, 85, 110, 350　　D. 60, 90, 110, 350

21. D

在 5 阶 B 树中，除根结点外的非叶结点的关键字数 k 需要满足 $2 \leqslant k \leqslant 4$。当被删关键字 x 不在终端结点（最底层非叶结点）时，可以用 x 的前驱（或后继）关键字 y 来替代 x，然后在相应结点中删除 y。情况①：删除 260，将其前驱 110 放入 260 处，删除 110 后的结点<100>不满足 5 阶 B 树定义，从左兄弟中借 85，将 85 放入根中，将根中的 90 移入结点<100>变为<90, 100>。情况②：删除 260，将其后继 280 放入 260 处，结点<300>不满足 5 阶 B 树定义且左右兄弟都不够借，结点<300>可以和左兄弟<100, 110>以及关键字 280 合并成一个新的结点<100, 110, 280, 300>。情况③：在情况②中，结点<300>也可以和右兄弟<400, 500>以及关键字 350 合并成一个新的结点<300, 350, 400, 500>。综上，T_1 根结点中的关键字序列可能是<60, 85, 110, 350>或<60, 90, 350>或<60, 90, 280>，仅 D 不可能。

快速解法：假如选项 D 的 60, 90, 110, 350 作为根结点，则在 90 和 110 之间只有 100 这一个数据，显然不符合 5 阶 B 树的定义，因此 D 项不可能。

16. 【2016 统考真题】B+树不同于 B 树的特点之一是（ ）。

A. 能支持顺序查找　　B. 结点中含有关键字

C. 根结点至少有两个分支　　D. 所有叶结点都在同一层上

16. A

由于 B+树的所有叶结点中包含了全部的关键字信息，且叶结点本身依关键字从小到大顺序链接，因此可以进行顺序查找，而 B 树不支持顺序查找（只支持多路查找）。

21. 【2019 统考真题】现有长度为 11 且初始为空的散列表 HT，散列函数是 $H(\text{key})=\text{key}\%7$，采用线性探查（线性探测再散列）法解决冲突。将关键字序列 87, 40, 30, 6, 11, 22, 98, 20 依次插入 HT 后，HT 查找失败的平均查找长度是（　）。

A. 4　　B. 5.25　　C. 6　　D. 6.29

21. C

采用线性探查法计算每个关键字的存放情况如下表所示。

散列地址	0	1	2	3	4	5	6	7	8	9	10
关键字	98	22	30	87	11	40	6	20			

由于 $H(\text{key})=0\sim6$，查找失败时可能对应的地址有 7 个，对于计算出地址为 0 的关键字 key0，只有比较完 0～8 号地址后才能确定该关键字不在表中，比较次数为 9；对于计算出地址为 1 的关键字 key1，只有比较完 1～8 号地址后才能确定该关键字不在表中，比较次数为 8；以此类推。需要特别注意的是，散列函数不可能计算出地址 7，因此有

$$\text{ASL}_{\text{失败}}=(9+8+7+6+5+4+3)/7=6$$

23.【2023 统考真题】现有长度为 5、初始为空的散列表 HT，散列函数 H(k)=(k+4)%5，用线性探查再散列法解决冲突。若将关键字序列 2022,12,25 依次插入 HT，然后删除关键字 25，则 HT 中查找失败的平均查找长度为（ ）。

A. 1　　B. 1.6　　C. 1.8　　D. 2.2

23. C

当采用开放定址法时，不能随便物理删除表中的已有元素，因为若删除元素，则可能截断其他具有相同散列地址的元素的查找地址。因此，当要删除一个元素时，可给它做一个删除标记。依次将 2022, 12, 25 插入散列表，然后删除 25，得到的散列表如下：

地址	0	1	2	3	4
关键字		2022	12		25（删除）
查找失败次数	1	3	2	1	2

当查找位置是删除标记时，应继续往后查找。

查找失败的平均查找长度为(1 + 3 + 2 + 1 + 2)/5 = 1.8。

11. 【2019 统考真题】设主串 T='abaabaabcabaabc'，模式串 S='abaabc'，采用 KMP 算法进行模式匹配，到匹配成功时为止，在匹配过程中进行的单个字符间的比较次数是(　)。

A. 9　　B. 10　　C. 12　　D. 15

11. B

假设位序从 0 开始的，按照 next 数组生成算法，对于 S 有

编号	0	1	2	3	4	5
S	a	b	a	a	b	c
next	-1	0	0	1	1	2

第一趟连续比较 6 次，在模式串的 5 号位和主串的 5 号位匹配失败，模式串的下一个比较位置为 next[5]，即下一次比较从模式串的 2 号位和主串的 5 号位开始，然后直到模式串的 5 号位和主串的 8 号位匹配，第二趟比较 4 次，匹配成功。单个字符的比较次数为 10 次。